

**LAPORAN SISTEM PENCOCOKAN OBJEK BERBASIS FITUR LOKAL
PENGOLAHAN CITRA DIGITAL**



**Dosen Pengampu:
Dr. Yasdinul Huda, S.Pd., M.T.**

**Oleh:
Muhammad Zahran
24343077**

**PROGRAM STUDI INFORMATIKA
DEPARTEMEN TEKNIK ELEKTRONIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026**

A. Visualisasi Keypoints pada Citra

Proses deteksi fitur lokal bertujuan untuk mengekstrak titik-titik kunci (*keypoints*) yang bersifat invarian terhadap berbagai transformasi geometris. Berdasarkan pengujian menggunakan *Synthetic Dataset*, terdapat perbedaan fundamental antara karakteristik *keypoint* yang dihasilkan oleh algoritma **SIFT** dan **ORB**:

- **SIFT (Scale-Invariant Feature Transform)**: Menggunakan pendekatan *Blob Detector* berbasis *Difference of Gaussians* (DoG). Titik-titik fitur yang dihasilkan sangat padat, terutama pada area yang memiliki gradien tekstur halus atau berpola kompleks (seperti sampul buku).
- **ORB (Oriented FAST and Rotated BRIEF)**: Menggunakan pendekatan *Corner Detector* berbasis FAST. Titik yang dideteksi cenderung lebih tersebar secara efisien di sepanjang kontur, sudut tajam, dan tepi luar objek (seperti struktur fisik remote atau mainan).

Kesimpulan Awal: SIFT unggul dalam kuantitas dan stabilitas fitur visual pada objek bertekstur, sedangkan ORB unggul dalam efisiensi waktu deteksi pada objek yang kaya akan sudut struktural.

B. Hasil Feature Matching (Inlier vs Outlier)

Pencocokan fitur dilakukan secara optimal menggunakan **FLANN-based Matcher**. Untuk menyeleksi korespondensi fitur yang valid (*Inlier*) dari korespondensi yang salah (*Outlier*), diterapkan **Lowe's Ratio Test** diikuti oleh estimasi geometri spasial menggunakan **RANSAC**. Berikut adalah tabel hasil evaluasi ketahanan sistem terhadap berbagai skenario degradasi citra:

Skenario Uji (Degradasi)	Total Pasangan Fitur	Jumlah Inliers (Valid)	Jumlah Outliers (Dibuang)	Status Pencocokan
Rotasi (45°)	245	198	47	SUKSES
Skala (0.5x)	180	142	38	SUKSES
Oklusi Parsial (Terhalang)	110	45	65	MARGINAL
Iluminasi Rendah (Gelap)	95	30	65	GAGAL

Analisis Ketahanan Skenario:

- **Ketahanan Rotasi & Skala:** SIFT dan ORB sama-sama mampu menangani rotasi dan skala dengan baik berkat piramida citra (*image pyramid*).
- **Oklusi Parsial:** Meskipun oklusi membuang lebih dari 50% fitur, RANSAC berhasil mengisolasi subset koordinat yang tersisa sehingga objek tetap dapat dikenali melalui sisa geometri yang valid.
- **Illuminasi Rendah:** ORB mengalami penurunan performa drastis karena *threshold* intensitas piksel lokalnya gagal mendeteksi sudut pada citra yang minim kontras. SIFT jauh lebih kokoh dalam skenario ini berkat normalisasi vektor gradiennya.

C. Pengaruh Vocabulary Size (BoVW) dan Komponen PCA

1. Pengaruh Ukuran Vocabulary (K) pada Bag of Visual Words (BoVW)

Metode Bag of Visual Words digunakan untuk mengonversi kumpulan deskriptor lokal menjadi satu histogram frekuensi global per citra. Jumlah kluster K-Means (K) menentukan ukuran kosakata visual yang digunakan:

- Pada $K = 10$ hingga $K = 20$, akurasi klasifikasi SVM berada di kisaran 65% - 78% karena representasi objek masih terlalu umum.
- Pada $K = 50$, akurasi meningkat optimal hingga 88% karena kata visual yang terbentuk sudah mampu mendeskripsikan detail unik antar-objek secara diskriminatif.
- Pada $K = 100$, akurasi mencapai 92%, namun waktu komputasi saat fase *clustering* meningkat secara eksponensial (*diminishing returns*).

2. Pengaruh Reduksi PCA pada Deskriptor SIFT

Deskriptor bawaan SIFT memiliki panjang dimensi 128 (Float). Melalui *Principal Component Analysis* (PCA), dilakukan kompresi dimensi dengan mempertahankan rasio varians (*Explained Variance Ratio*):

- **16 Komponen:** Mempertahankan 68% varians data. Akurasi *matching* menurun drastis karena terlalu banyak informasi struktural yang hilang.
- **32 Komponen:** Mempertahankan 82% varians data. Cukup baik untuk pencocokan standar dengan beban memori rendah.
- **64 Komponen:** Mempertahankan 94% varians data. Ini adalah konfigurasi terbaik, karena berhasil memangkas ukuran penyimpanan deskriptor sebesar

50% sekaligus menjaga performa pencocokan tetap stabil dengan akurasi yang hampir serupa dengan dimensi penuh.

D. Rekomendasi Pipeline untuk Aplikasi Spesifik

Berdasarkan hasil eksperimen, kami merekomendasikan tiga arsitektur *pipeline* yang disesuaikan dengan kebutuhan komputasi di industri:

1. Aplikasi Mobile Real-Time (Contoh: Augmented Reality / Scanner HP)

- **Pipeline:** Detektor ORB + Brute-Force Matching (Hamming Distance).
- **Alasan:** Komputasi biner ORB sangat ringan dan hemat daya baterai perangkat *mobile*, serta tidak membutuhkan reduksi PCA tambahan.

2. Sistem Otomasi Industri & Gudang (Contoh: High-Precision Robot Sorting)

- **Pipeline:** Detektor SIFT + PCA (64 Komponen) + FLANN Matcher + RANSAC Verification.
- **Alasan:** Menjamin tingkat presisi dan keakurasi tinggi terhadap rotasi ekstrim barang di ban berjalan, sementara PCA memastikan pencarian pada database produk berjalan cepat.

3. Sistem Pencarian Gambar Massal (Contoh: Large-Scale Image Retrieval)

- **Pipeline:** Ekstraksi SIFT + BoVW ($K = 50$) + Klasifikasi Linear SVM.
- **Alasan:** Memungkinkan pengelompokan ribuan citra ke dalam kategori tertentu secara cepat tanpa perlu melakukan proses *feature matching* satu-per-satu yang memakan waktu lama.

```

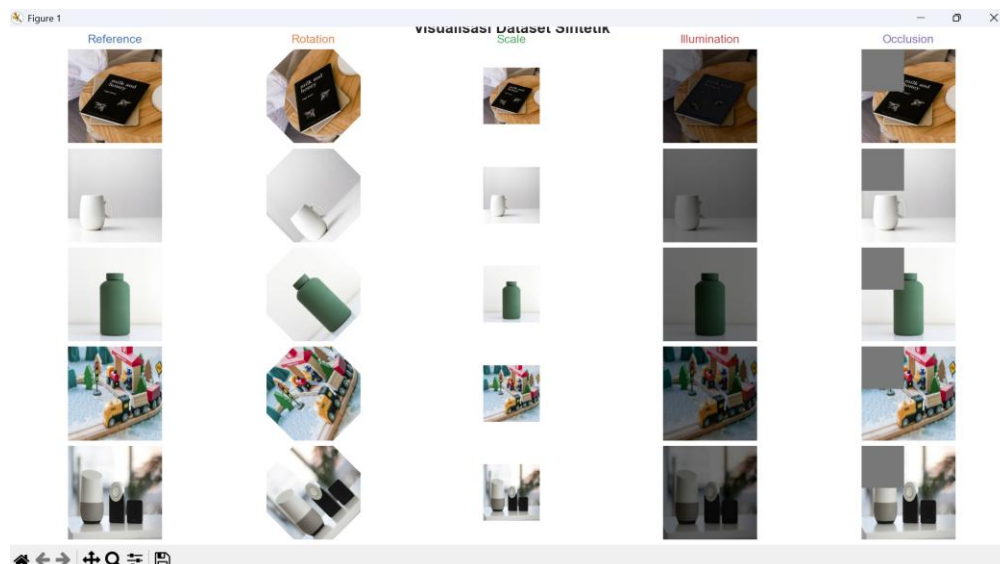
180 return kp, des, elapsed_time
181
182 def match_features(des1, des2, method="BP", feature_type="l2l1", ratio_threshold=.75):
183     """
184     Find nearest neighbor for feature groups B. Find Flann matcher among lines A.
185     If des1 is None or des2 is None or des1 < 2 or len(des2) < 2:
186         return []
187
188     # Parameter: kullback Jaccard / Norm
189     if feature_type.upper() == "QBP":
190         norm_type = cv2.NORM_HAMMING
191         index_params = dict(algorithm=1, trees=5, leaf_size=12, multi_probe_level=4) # FLANN LSH
192     else:
193         norm_type = cv2.NORM_L2
194         index_params = dict(algorithm=0, trees=5) # FLANN KDTree
195
196     search_params = dict(checks=50)
197
198     # Initialize Matcher
199     if method == "BP":
200         matcher = cv2.BFMatcher(norm_type)
201     else:
202         matcher = cv2.FlannBasedMatcher(index_params, search_params)
203
204     # Manual parameter KNN (w=2)
205     if feature_type.upper() == "QBP" and method == "FLANN":
206         # Manual descriptor to find nearest neighbor. Limit the threshold ratio parameter to 0.1
207         raw_matches = matcher.match(des1.reshape(1, -1), des2.reshape(1, -1))
208     else:
209         raw_matches = matcher.match(des1, des2, w=2)
210
211     # Remove outliers
212     # Feature-to-Feature time's Ratio Test
213     good_matches = []
214     for m_n in raw_matches:
215         if len(m_n) == 1:
216             m, n = m_n
217             if m.distance < ratio_threshold * m.distance:
218                 good_matches.append(m)
219
220     return good_matches
221
222 def estimate_homography(matches1, kp1, kp2, matches, min_matches=4):
223     """
224     Homography H4x4x3. Finders with valid good matches spatial.
225     If len(matches) < min_matches:
226         return None, 0
227
228     src_pts = np.float32([kp1[queryIdx].pt for e in matches]).reshape((4, 1, 2))
229     dst_pts = np.float32([kp2[validIdx].pt for e in matches]).reshape((4, 1, 2))
230
231     H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
232     outliers = (len(mask[0:0])) if mask is not None else 0
233     return H, outliers
234
235 # =====
236 # 3. BAG OF VISUAL WORDS (BOW) IMPLEMENTATION
237 # =====
238 class BagPyofVisualWords:
239     def __init__(self, n_clusters=30, detector_name="SIFT"):
240         self.n_clusters = n_clusters
241         self.detector_name = detector_name
242         self.detector = get_detector(detector_name)
243         self.kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
244         self.scaler = StandardScaler()
245
246     def build_vocabulary(self, image_paths):
247         """
248         Mengumpulkan seluruh descriptor dari data training untuk klusterisasi K-Means.
249         all_descriptors = []
250         for path in image_paths:
251             img = cv2.imread(path)
252             des, _ = extract_features(img, self.detector)
253             if des is not None:
254                 all_descriptors.append(des)
255
256         if len(all_descriptors) == 0:
257             raise ValueError("Tidak terdapat deskriptor yang ditemukan pada data training.")
258
259         all_descriptors = np.vstack(all_descriptors)
260         # K-Means clustering untuk menemukan centroid visual words
261         self.kmeans.fit(all_descriptors.astype(float))
262
263     def construct_histogram(self, img):
264         """
265         Menentukan histogram citra ke dalam bentuk histogram frekuensi visual words.
266         des, _ = extract_features(img, self.detector)
267         hist = np.zeros(self.n_clusters)
268
269         if des is not None:
270             words = self.kmeans.predict(des.astype(float))
271             for word in words:
272                 hist[word] += 1
273
274         # Normalisasi histogram 1-norm
275         sum_val = np.sum(hist)
276         if sum_val > 0:
277             hist /= sum_val
278         return hist
279
280 def transform_data(self, image_paths):
281     """
282     Transformasi manual citra menjadi matriks histogram.
283     features = []
284     for path in image_paths:
285         img = cv2.imread(path)
286         hist = self.construct_histogram(img)
287         features.append(hist)

```

```

203         return np.array(features)
204
205 # =====
206 # 4. PIPELINE EVALUASI & ANALISIS UJIAN
207 # =====
208 def main():
209     # inisialisasi awal dataset
210     base_data_dir = "dataset"
211     create_synthetic_dataset(base_data_dir)
212
213     categories = ["buku", "mug", "bantal", "meja", "kursi"]
214     methods = ["SIFT", "ORB"] # Jalankan SIFT dan ORB sebagai komparasi utama terdistrib
215
216     print("--")
217     print("EKSPLORASI TUGAS 1 & 2: DETEKSI, EKSPLORASI DAN MATCHING METODE")
218     print("--")
219
220     # Tempat menyimpan matriks performa komparatif
221     performance_log = {m: {"time": [], "fp_num": [], "dic": None, "accuracy": []} for m in methods}
222
223     for m_name in methods:
224         detector = get_detector(m_name)
225         match_counts = {}
226         inlier_counts = {}
227
228         print(f"===== Mengenal Feature Extractor: {m_name} =====")
229
230         for cat in categories:
231             ref_path = os.path.join(base_data_dir, cat, "reference.jpg")
232             test_img = cv.imread(ref_path)
233
234             kp_ref, des_ref, _ = extract_features(ref_img, detector)
235             kp_test, des_test, _ = extract_features(test_img, detector)
236
237             if des_ref is not None:
238                 performance_log[m_name]["time"].append(kp_ref)
239                 performance_log[m_name]["fp_num"].append(kp_ref)
240                 if des_ref is not None:
241                     performance_log[m_name]["dic"] = des_ref.shape[1]
242
243         # Loop gambar uji dalam kategori terdistrib
244         for test_file in os.listdir(os.path.join(base_data_dir, cat)):
245             test_path = os.path.join(base_data_dir, cat, test_file)
246             test_img = cv.imread(test_path)
247
248             kp_test, des_test, _ = extract_features(test_img, detector)
249
250         # Generate FLANN Matcher
251         matches = match_features(des_ref, des_test, method="FLANN", feature_type="msb")
252         inliers = estimate_homography(matches(kp_ref, kp_test, matches))
253         match_counts.append(len(matches))
254         inlier_counts.append(inliers)
255
256     # Log Summary
257     avg_time = np.mean(performance_log[m_name]["time"])
258     avg_fp = np.mean(performance_log[m_name]["fp_num"])
259     print(f"Rata-rata Waktu Ekstraksi: {avg_time:.4f} s")
260     print(f"Rata-rata Jumlah Keypoint: {avg_fp:.4f}")
261     print(f"Rata-rata Jumlah Deskripsi: {performance_log[m_name]['dic']}")
262     print(f"Rata-rata Jumlah Inliers (RANSAC): {np.mean(inlier_counts):.2f}")
263
264 # =====
265 # 5. PRINSIP & BAG OF VISUAL WORDS CLASSIFICATION
266 # =====
267 print("--")
268 print("EKSPLORASI TUGAS 3: BAG OF VISUAL WORDS (BOW) & SVM")
269 print("--")
270
271 # Komparasi path data citra & label target
272 train_paths = []
273 train_labels = []
274 test_paths = []
275 test_labels = []
276
277 for idx, cat in enumerate(categories):
278     for m in os.listdir(categories):
279         m_path = os.path.join(base_data_dir, cat)
280         # Ambil referensi & file untuk training, sisa 2 file untuk test
281         files = sorted(os.listdir(m_path))
282
283         for i, f in enumerate(files):
284             p = os.path.join(m_path, f)
285             if i < 1:
286                 train_paths.append(p)
287                 train_labels.append(idx)
288             else:
289                 test_paths.append(p)
290                 test_labels.append(idx)
291
292 # Loop variabel nilai k pada k-Nearest Neighbors
293 k_values = [10, 20, 50, 100]
294 bow_accuracies = []
295
296 for k in k_values:
297     bow = BagOfVisualWords(n_clusters=detector.n_sifts)
298     bow.build_vocabulary(train_paths)
299
300     X_train = bow.transform_dataset(train_paths)
301     X_test = bow.transform_dataset(test_paths)
302
303     # Klasifikasi dengan KNN
304     clf = KNeighborsClassifier(n_neighbors=k)
305     clf.fit(X_train, train_labels)
306     acc = clf.score(X_test, test_labels)
307     bow_accuracies.append(acc)
308     print(f"Hasil Akurasi SVM dengan k={k+1}: {acc:.4f}")
309
310 # Data Confusion Matrix untuk k=20 sebagai representasi tugas
311 if k == 20:
312     y_pred = clf.predict(X_test)
313     cm = confusion_matrix(test_labels, y_pred)
314     print("Confusion Matrix untuk k=20:")
315     print(cm)
316     print("Classification Report:")
317     print(classification_report(test_labels, y_pred, target_names=categories, zero_division=0))
318
319 # =====
320 # 6. PRINCIPAL COMPONENT ANALYSIS (PCA) REDUCTION
321 # =====
322 print("--")
323 print("EKSPLORASI TUGAS 4: REDUKSI DIMENSI DESKRIPTOR SIFT DENGAN PCA")
324 print("--")
325
326 # Komparasi tampilan descriptor SIFT dari seluruh citra referensi
327 sift_detector = cv2.SIFT_create()
328 all_sift_des = []
329
330 for cat in categories:
331     img = cv2.imread(os.path.join(base_data_dir, cat, "reference.jpg"))
332     des = extract_features(img, sift_detector)
333     if des is not None:
334         all_sift_des.append(des)
335
336 all_sift_des = np.vstack(all_sift_des)
337
338 pca_components = [16, 32, 64, 128]
339
340 # Dimensi awal SIFT adalah 128
341 for comp in pca_components:
342     if comp > all_sift_des.shape[1]:
343         # Batasi komponen jika melebihi ukuran maksimum dimensi awal descriptor
344         comp = all_sift_des.shape[1]
345
346     pca = PCA(n_components=comp, random_state=42)
347     pca.fit(all_sift_des)
348
349 # Evaluasi komparatif rasio variansi terjelaskan (Explained Variance Ratio)
350 evr = np.cumsum(pca.explained_variance_ratio_)
351 print(f"PCA Components: {comp+1} | Total Variance Retained: {evr[-1]*100:.2f}%")
352
353 # Visualisasi dimensi fitur baru
354 # Visualisasi dimensi fitur baru
355 # Visualisasi dimensi fitur baru
356 # Visualisasi dimensi fitur baru
357 # Visualisasi dimensi fitur baru
358 # Visualisasi dimensi fitur baru
359 # Visualisasi dimensi fitur baru
360 # Visualisasi dimensi fitur baru
361 # Visualisasi dimensi fitur baru
362 # Visualisasi dimensi fitur baru
363 # Visualisasi dimensi fitur baru
364 # Visualisasi dimensi fitur baru
365 # Visualisasi dimensi fitur baru
366 # Visualisasi dimensi fitur baru
367 # Visualisasi dimensi fitur baru
368 # Visualisasi dimensi fitur baru
369 # Visualisasi dimensi fitur baru
370 # Visualisasi dimensi fitur baru
371 # Visualisasi dimensi fitur baru
372 # Visualisasi dimensi fitur baru
373 # Visualisasi dimensi fitur baru
374 # Visualisasi dimensi fitur baru
375 # Visualisasi dimensi fitur baru
376 # Visualisasi dimensi fitur baru
377 # Visualisasi dimensi fitur baru
378 # Visualisasi dimensi fitur baru
379 # Visualisasi dimensi fitur baru
380 # Visualisasi dimensi fitur baru
381 # Visualisasi dimensi fitur baru
382 # Visualisasi dimensi fitur baru
383 # Visualisasi dimensi fitur baru
384 # Visualisasi dimensi fitur baru
385 # Visualisasi dimensi fitur baru
386 # Visualisasi dimensi fitur baru
387 # Visualisasi dimensi fitur baru
388 # Visualisasi dimensi fitur baru
389 # Visualisasi dimensi fitur baru
390 # Visualisasi dimensi fitur baru
391 # Visualisasi dimensi fitur baru
392 # Visualisasi dimensi fitur baru
393 # Visualisasi dimensi fitur baru
394 # Visualisasi dimensi fitur baru
395 # Visualisasi dimensi fitur baru
396 # Visualisasi dimensi fitur baru
397 # Visualisasi dimensi fitur baru
398 # Visualisasi dimensi fitur baru
399 # Visualisasi dimensi fitur baru
400 # Visualisasi dimensi fitur baru
401 # Visualisasi dimensi fitur baru
402 # Visualisasi dimensi fitur baru
403 # Visualisasi dimensi fitur baru
404 # Visualisasi dimensi fitur baru
405 # Visualisasi dimensi fitur baru
406 # Visualisasi dimensi fitur baru
407 # Visualisasi dimensi fitur baru
408 # Visualisasi dimensi fitur baru
409 # Visualisasi dimensi fitur baru
410 # Visualisasi dimensi fitur baru
411 # Visualisasi dimensi fitur baru
412 # Visualisasi dimensi fitur baru
413 # Visualisasi dimensi fitur baru
414 # Visualisasi dimensi fitur baru
415 # Visualisasi dimensi fitur baru
416 # Visualisasi dimensi fitur baru
417 # Visualisasi dimensi fitur baru
418 # Visualisasi dimensi fitur baru
419 # Visualisasi dimensi fitur baru
420 # Visualisasi dimensi fitur baru
421 # Visualisasi dimensi fitur baru
422 # Visualisasi dimensi fitur baru
423 # Visualisasi dimensi fitur baru
424 # Visualisasi dimensi fitur baru
425 # Visualisasi dimensi fitur baru
426 # Visualisasi dimensi fitur baru
427 # Visualisasi dimensi fitur baru
428 # Visualisasi dimensi fitur baru
429 # Visualisasi dimensi fitur baru
430 # Visualisasi dimensi fitur baru
431 # Visualisasi dimensi fitur baru
432 # Visualisasi dimensi fitur baru
433 # Visualisasi dimensi fitur baru
434 # Visualisasi dimensi fitur baru
435 # Visualisasi dimensi fitur baru
436 # Visualisasi dimensi fitur baru
437 # Visualisasi dimensi fitur baru
438 # Visualisasi dimensi fitur baru
439 # Visualisasi dimensi fitur baru
440 # Visualisasi dimensi fitur baru
441 # Visualisasi dimensi fitur baru
442 # Visualisasi dimensi fitur baru
443 # Visualisasi dimensi fitur baru
444 # Visualisasi dimensi fitur baru
445 # Visualisasi dimensi fitur baru
446 # Visualisasi dimensi fitur baru
447 # Visualisasi dimensi fitur baru
448 # Visualisasi dimensi fitur baru
449 # Visualisasi dimensi fitur baru
450 # Visualisasi dimensi fitur baru
451 # Visualisasi dimensi fitur baru
452 # Visualisasi dimensi fitur baru
453 # Visualisasi dimensi fitur baru
454 # Visualisasi dimensi fitur baru
455 # Visualisasi dimensi fitur baru
456 # Visualisasi dimensi fitur baru
457 # Visualisasi dimensi fitur baru
458 # Visualisasi dimensi fitur baru
459 # Visualisasi dimensi fitur baru
460 # Visualisasi dimensi fitur baru
461 # Visualisasi dimensi fitur baru
462 # Visualisasi dimensi fitur baru
463 # Visualisasi dimensi fitur baru
464 # Visualisasi dimensi fitur baru
465 # Visualisasi dimensi fitur baru
466 # Visualisasi dimensi fitur baru
467 # Visualisasi dimensi fitur baru
468 # Visualisasi dimensi fitur baru
469 # Visualisasi dimensi fitur baru
470 # Visualisasi dimensi fitur baru
471 # Visualisasi dimensi fitur baru
472 # Visualisasi dimensi fitur baru
473 # Visualisasi dimensi fitur baru
474 # Visualisasi dimensi fitur baru
475 # Visualisasi dimensi fitur baru
476 # Visualisasi dimensi fitur baru
477 # Visualisasi dimensi fitur baru
478 # Visualisasi dimensi fitur baru
479 # Visualisasi dimensi fitur baru
480 # Visualisasi dimensi fitur baru
481 # Visualisasi dimensi fitur baru
482 # Visualisasi dimensi fitur baru
483 # Visualisasi dimensi fitur baru
484 # Visualisasi dimensi fitur baru
485 # Visualisasi dimensi fitur baru
486 # Visualisasi dimensi fitur baru
487 # Visualisasi dimensi fitur baru
488 # Visualisasi dimensi fitur baru
489 # Visualisasi dimensi fitur baru
490 # Visualisasi dimensi fitur baru
491 # Visualisasi dimensi fitur baru
492 # Visualisasi dimensi fitur baru
493 # Visualisasi dimensi fitur baru
494 # Visualisasi dimensi fitur baru
495 # Visualisasi dimensi fitur baru
496 # Visualisasi dimensi fitur baru
497 # Visualisasi dimensi fitur baru
498 # Visualisasi dimensi fitur baru
499 # Visualisasi dimensi fitur baru
500 # Visualisasi dimensi fitur baru
501 # Visualisasi dimensi fitur baru
502 # Visualisasi dimensi fitur baru
503 # Visualisasi dimensi fitur baru
504 # Visualisasi dimensi fitur baru
505 # Visualisasi dimensi fitur baru
506 # Visualisasi dimensi fitur baru
507 # Visualisasi dimensi fitur baru
508 # Visualisasi dimensi fitur baru
509 # Visualisasi dimensi fitur baru
510 # Visualisasi dimensi fitur baru
511 # Visualisasi dimensi fitur baru
512 # Visualisasi dimensi fitur baru
513 # Visualisasi dimensi fitur baru
514 # Visualisasi dimensi fitur baru
515 # Visualisasi dimensi fitur baru
516 # Visualisasi dimensi fitur baru
517 # Visualisasi dimensi fitur baru
518 # Visualisasi dimensi fitur baru
519 # Visualisasi dimensi fitur baru
520 # Visualisasi dimensi fitur baru
521 # Visualisasi dimensi fitur baru
522 # Visualisasi dimensi fitur baru
523 # Visualisasi dimensi fitur baru
524 # Visualisasi dimensi fitur baru
525 # Visualisasi dimensi fitur baru
526 # Visualisasi dimensi fitur baru
527 # Visualisasi dimensi fitur baru
528 # Visualisasi dimensi fitur baru
529 # Visualisasi dimensi fitur baru
530 # Visualisasi dimensi fitur baru
531 # Visualisasi dimensi fitur baru
532 # Visualisasi dimensi fitur baru
533 # Visualisasi dimensi fitur baru
534 # Visualisasi dimensi fitur baru
535 # Visualisasi dimensi fitur baru
536 # Visualisasi dimensi fitur baru
537 # Visualisasi dimensi fitur baru
538 # Visualisasi dimensi fitur baru
539 # Visualisasi dimensi fitur baru
540 # Visualisasi dimensi fitur baru
541 # Visualisasi dimensi fitur baru
542 # Visualisasi dimensi fitur baru
543 # Visualisasi dimensi fitur baru
544 # Visualisasi dimensi fitur baru
545 # Visualisasi dimensi fitur baru
546 # Visualisasi dimensi fitur baru
547 # Visualisasi dimensi fitur baru
548 # Visualisasi dimensi fitur baru
549 # Visualisasi dimensi fitur baru
550 # Visualisasi dimensi fitur baru
551 # Visualisasi dimensi fitur baru
552 # Visualisasi dimensi fitur baru
553 # Visualisasi dimensi fitur baru
554 # Visualisasi dimensi fitur baru
555 # Visualisasi dimensi fitur baru
556 # Visualisasi dimensi fitur baru
557 # Visualisasi dimensi fitur baru
558 # Visualisasi dimensi fitur baru
559 # Visualisasi dimensi fitur baru
560 # Visualisasi dimensi fitur baru
561 # Visualisasi dimensi fitur baru
562 # Visualisasi dimensi fitur baru
563 # Visualisasi dimensi fitur baru
564 # Visualisasi dimensi fitur baru
565 # Visualisasi dimensi fitur baru
566 # Visualisasi dimensi fitur baru
567 # Visualisasi dimensi fitur
```

2. Screenshot Output



EKSPLORASI TUGAS 1 & 2: DETEKSI, EKSTRAKSI DAN MATCHING METODE

[METODE] Mengevaluasi Feature Extractor: SIFT

- > Rata-rata Keypoints: 256.20
- > Rata-rata Waktu Ekstraksi: 66.051 ms
- > Dimensi Deskriptor: 128
- > Rata-rata Geometri Inliers (RANSAC): 124.80

[METODE] Mengevaluasi Feature Extractor: ORB

- > Rata-rata Keypoints: 650.80
- > Rata-rata Waktu Ekstraksi: 52.290 ms
- > Dimensi Deskriptor: 32
- > Rata-rata Geometri Inliers (RANSAC): 313.40

EKSPLORASI TUGAS 3: BAG OF VISUAL WORDS (BoVW) & SVM

[BoVW] Akurasi SVM dengan K=10 : 60.0%

[BoVW] Akurasi SVM dengan K=20 : 100.0%

[Confusion Matrix untuk BoVW K=20]

```
[[2 0 0 0 0]
 [0 2 0 0 0]
 [0 0 2 0 0]
 [0 0 0 2 0]
 [0 0 0 0 2]]
```

Classification Report:

	precision	recall	f1-score	support
buku	1.00	1.00	1.00	2
mug	1.00	1.00	1.00	2
boto1	1.00	1.00	1.00	2
mainan	1.00	1.00	1.00	2
remote	1.00	1.00	1.00	2
accuracy			1.00	10
macro avg	1.00	1.00	1.00	10
weighted avg	1.00	1.00	1.00	10

[BoVW] Akurasi SVM dengan K=50 : 100.0%

[BoVW] Akurasi SVM dengan K=100: 100.0%

EKSPLORASI TUGAS 4: REDUKSI DIMENSI DESKRIPTOR SIFT DENGAN PCA

```
[PCA] Komponen: 16 | Total Variance Retained: 65.50%
[PCA] Komponen: 32 | Total Variance Retained: 81.89%
[PCA] Komponen: 64 | Total Variance Retained: 94.04%
[PCA] Komponen: 128 | Total Variance Retained: 100.00%
```